ENTERPRISE DEVSECOPS ADOPTION STRATEGY

DevSecOps Rollout Plan for 40 Microservices

Project:	DevSecOps Security Controls Rollout for OWASP Juice Shop	**Prepared By:**	Sandeep Sain
Role:	Application Security Engineer	**Date:**	31 July 2026

EXECUTIVE SUMMARY

This strategy outlines the phased rollout of automated DevSecOps security controls across our 40 microservices. The objective is to systematically reduce technical risk, prevent secret leaks, and eliminate high-severity vulnerabilities without bottlenecking developer velocity or halting product releases.

PHASED IMPLEMENTATION STRATEGY

Phase 1 – Pilot Implementation (Weeks 1–2)

Objective: Validate the pipeline controls on high-value, high-risk microservices prior to full org adoption.

Target Pilot Services: Authentication Service, Payment Service, API Gateway, Customer Profile Service, Admin Portal.

- **Selection Rationale:** Handle sensitive customer PII, process financial transactions, control access, and face the public Internet.
- **Controls Deployed:** SAST (Semgrep), Secrets Scan (Gitleaks), SCA & SBOM (Trivy), Hardened Non-Root Dockerfiles.
- **Pipeline Mode:** Non-blocking advisory mode (generates reports without breaking builds unless Critical issues occur).

Phase 2 – Department Expansion (Weeks 3–5)

Objective: Scale controls to 15–20 intermediate microservices following pilot validation.

Expanded Services: Order Service, Inventory, Product Catalog, Notification, Reporting, Search, User Preferences.

- **Security Enforcement:** High and Critical findings block PR merges; hardcoded secrets halt builds immediately.
- **Developer Enablement:** Weekly security office hours, remediation guides, and inline code snippet examples.

Phase 3 – Organization-Wide Adoption (Weeks 6–8)

Objective: Standardize security baselines across all 40 microservices.

- **Mandatory Controls:** Branch protection (required 1 PR review), mandatory SAST/SCA/Secret checks, automated SBOM generation, hardened base container usage across all repositories.

DEFINING "CRITICAL SERVICES" & ADOPTION MODEL

Criteria for Critical Services

- Processes or routes financial/payment data
- Stores or transmits customer PII
- Manages authentication, tokens, or session state
- Exposes public-facing Internet endpoints
- Provides administrative or system-level access

Gradual Adoption Lifecycle

- **1. Monitor Mode:** Scans run passively; reports uploaded to PRs without blocking builds.
- **2. Warning Mode:** Scanner findings trigger PR warnings and developer training prompts.
- **3. Enforcement Mode:** Strict gating activated; High/Critical findings block pull requests.

EXCEPTION MANAGEMENT & HANDLING PUSHBACK

Structured Exception Workflow:

- Developers submit exception requests detailing the finding and technical/business constraints.
- AppSec team reviews the risk; approved exceptions receive a mandatory 30-day expiration timer.
- False positives are addressed by updating custom Semgrep rules or adding inline comments (e.g., `// semgrep-ignore`).

Addressing Developer Resistance:

- Enforce a strict 2-hour SLA for AppSec review on disputed pipeline blocks.
- Focus initial enforcement strictly on High and Critical vulnerabilities to avoid noise.
- Run intensive parallel scans to ensure CI/CD execution time increases by less than 60 seconds.

KEY OPERATIONAL METRICS (KPIS)

PERFORMANCE METRIC	TARGET THRESHOLD	OPERATIONAL GOAL
Microservices Onboarded	40 / 40 (100%)	Complete coverage across all active codebases.
Pipeline Build Pass Rate	> 95%	Minimize pipeline disruption and developer blockages.
Critical Vulnerability Remediation	100% FIXED	Zero Critical issues permitted in production builds.
Mean Time to Remediation (MTTR)	< 7 DAYS	Rapid patching cycle for flagged security flaws.
Pre-Merge Secret Detection	100% BLOCKED	Prevent credentials from ever entering git history.
Scanner False Positive Rate	< 10%	Continuously tune SAST/SCA rules to reduce noise.
SBOM & Container Scan Coverage	100% COVERAGE	Full inventory and container vulnerability visibility.

RISK MITIGATION & EXPECTED OUTCOMES

- **CI/CD Build Delays:** Mitigated by executing SAST, SCA, and Container scans in parallel jobs.
- **Legacy App Compatibility:** Addressed via temporary risk acceptances and phased monitor modes.
- **Target Outcome:** Standardized, automated DevSecOps governance across all 40 microservices, preventing secrets leakage and vulnerable dependencies from entering production.